МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 15

Тема: Поведенческие паттерны: Идиоматические паттерны

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная
инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы идиоматического паттерна «Producer-Consumer» и реализовать прикладную задачу «Графический пароль» с его использованием.

Задание: разработать приложение для работы с графическим паролем и реализовать один из паттернов: Producer-Consumer, Thread Pool, Future / Promise.

Выбран паттерн Producer-Consumer.

Вариант 8 - Графический пароль

1. Описание паттерна «Producer-Consumer»

Producer-Consumer - идиоматический паттерн многопоточности, при котором один поток (producer) генерирует задачи и помещает их в очередь, а другой (consumer) извлекает задачи из очереди и выполняет их. Потоки общаются через потокобезопасную очередь (queue.Queue в Python), что исключает гонки данных и необходимость ручной синхронизации.

Применяется, когда:

- нужно разгрузить главный поток (UI), чтобы интерфейс оставался отзывчивым;
- операции длительные (I/O, хэширование, сетевые вызовы);
- требуется буферизация задач между производителем и потребителем.

В приложении:

- UI (ProducerConsumerApp) - producer: формирует Task и кладёт в task_queue;
- Worker - consumer: в цикле читает task_queue.get() и выполняет задачу;
- result_queue - обратная очередь для передачи результатов в UI;
- UI опрашивает result_queue через root.after(50, ...) и обновляет элементы.

2. Описание разработанного приложения

Приложение позволяет формировать, сохранять, проверять и сбрасывать графический пароль. Все операции (create/verify/reset/status) выполняются в фоновом потоке.

Основное окно содержит:

- бейдж состояния сервиса;
- сетку 3×3;
- текущую последовательность;
- кнопки «Войти» / «Сохранить пароль», «Очистить»;
- переключатель «Вход / Создать пароль»;
- кнопку «Сбросить сохранённый пароль»;
- статусную строку;
- индикатор длины очереди задач.

Требования к паролю: длина 4–9, без повторов, ячейки 1–9.

3. Диаграмма классов

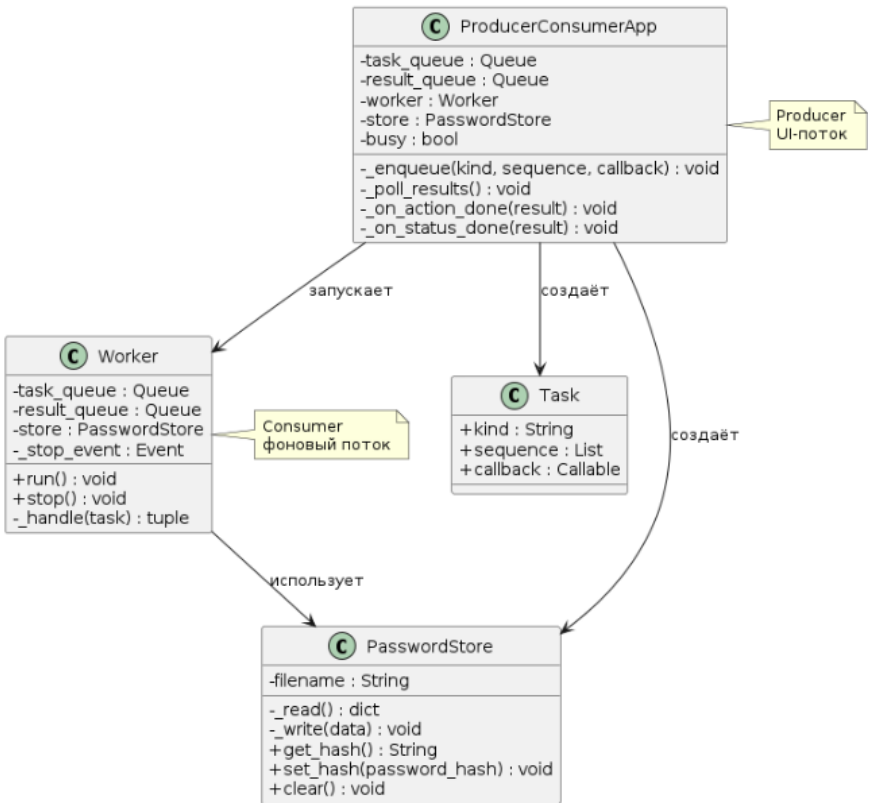


Рисунок 1 - Диаграмма классов приложения «Графический пароль» (Producer-Consumer)

4. Реализация паттерна «Producer-Consumer»

Очереди и поток-потребитель:

```
class Worker(threading.Thread):
    def __init__(self, task_queue, result_queue, store):
        super().__init__(daemon=True)
        self.task_queue = task_queue
        self.result_queue = result_queue
        self.store = store
        self._stop_event = threading.Event()

    def stop(self):
        self._stop_event.set()
        self.task_queue.put(None)

    def run(self):
        while not self._stop_event.is_set():
            task = self.task_queue.get()
            if task is None:
                break
            result = self._handle(task)
            self.result_queue.put((task.callback, result))
            self.task_queue.task_done()
```

Producer в UI:

```
def _enqueue(self, kind, sequence, callback):
    self.busy = True
    self.primary_btn.set_text("Обработка...")
    self.task_queue.put(Task(kind, sequence, callback))
    self._refresh()

def _poll_results(self):
```

```
    try:
        while True:
            callback, result = self.result_queue.get_nowait()
            callback(result)
    except queue.Empty:
        pass
    self.root.after(50, self._poll_results)
```

_poll_results вызывается из главного потока Tkinter и безопасно применяет результат: обновляет метки, показывает диалоги, разблокирует кнопки.

Обработка задачи в consumer:

```
def _handle(self, task):
    time.sleep(0.15)  # имитация длительной операции
    if task.kind == "create":
        ok, msg = validate_sequence(task.sequence)
        if not ok:
            return False, msg
        if self.store.get_hash() is not None:
            return False, "Пароль уже существует."
        self.store.set_hash(hash_sequence(task.sequence))
        return True, "Пароль сохранён."

    if task.kind == "verify":
        ok, msg = validate_sequence(task.sequence)
        if not ok:
            return False, msg
        stored = self.store.get_hash()
        if stored is None:
            return False, "Пароль не создан."
        if hash_sequence(task.sequence) == stored:
```

```
            return True, "Пароль верный. Доступ разрешён."
        return False, "Пароль неверный. Доступ запрещён."

    if task.kind == "reset":
        self.store.clear()
        return True, "Пароль удалён."

    if task.kind == "status":
        return True, "OK" if self.store.get_hash() else "Пароль не создан."
```

Ключевые моменты:

- оба потока общаются только через queue.Queue;
- Tkinter обновляется только из главного потока (_poll_results);
- daemon=True и stop() обеспечивают корректное завершение при закрытии окна.

5. Взаимодействие компонентов

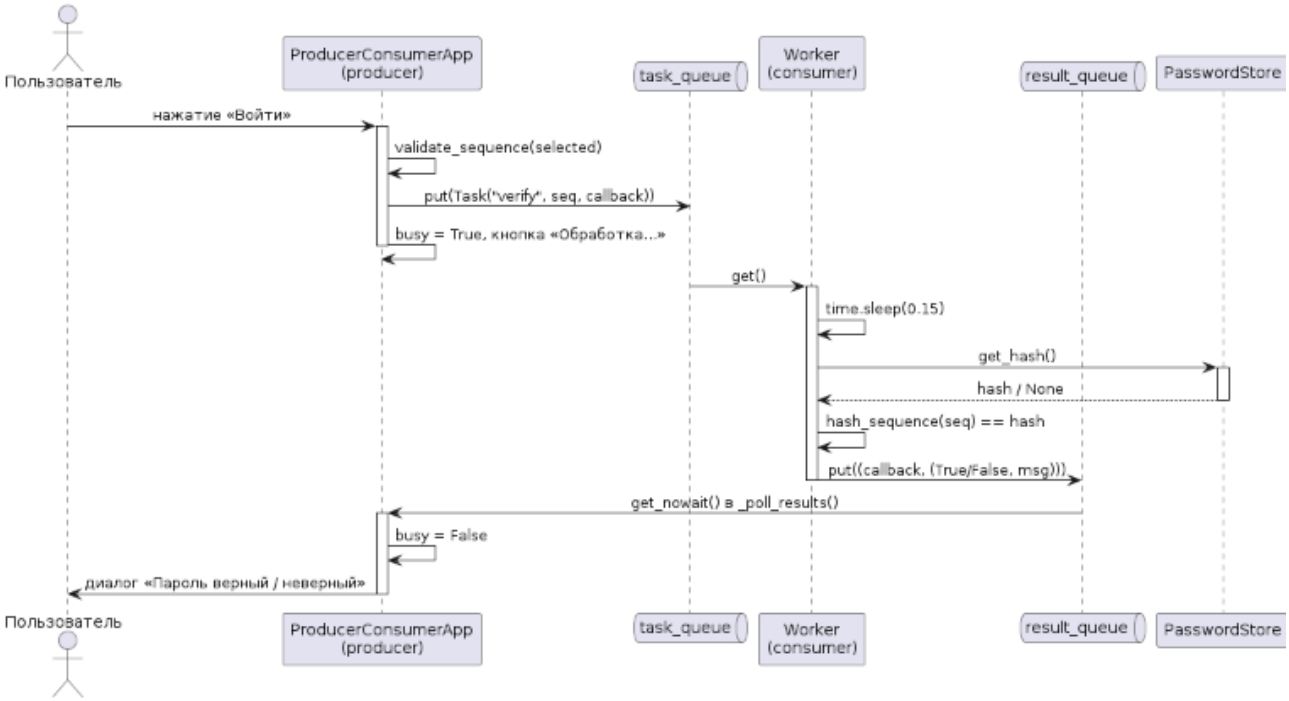


Рисунок 2 - Диаграмма последовательности для сценария «Войти»

Поток «Создать пароль» аналогичен, но worker вызывает store.set_hash(...).

Таблица 1 - Типы задач

kind	Действие worker	Результат
create	Валидация + запись хэша	«Пароль сохранён»
verify	Сравнение хэшей	«Пароль верный / неверный»
reset	Удаление файла	«Пароль удалён»
status	Проверка наличия	«OK / Пароль не создан»

6. Результаты работы программы

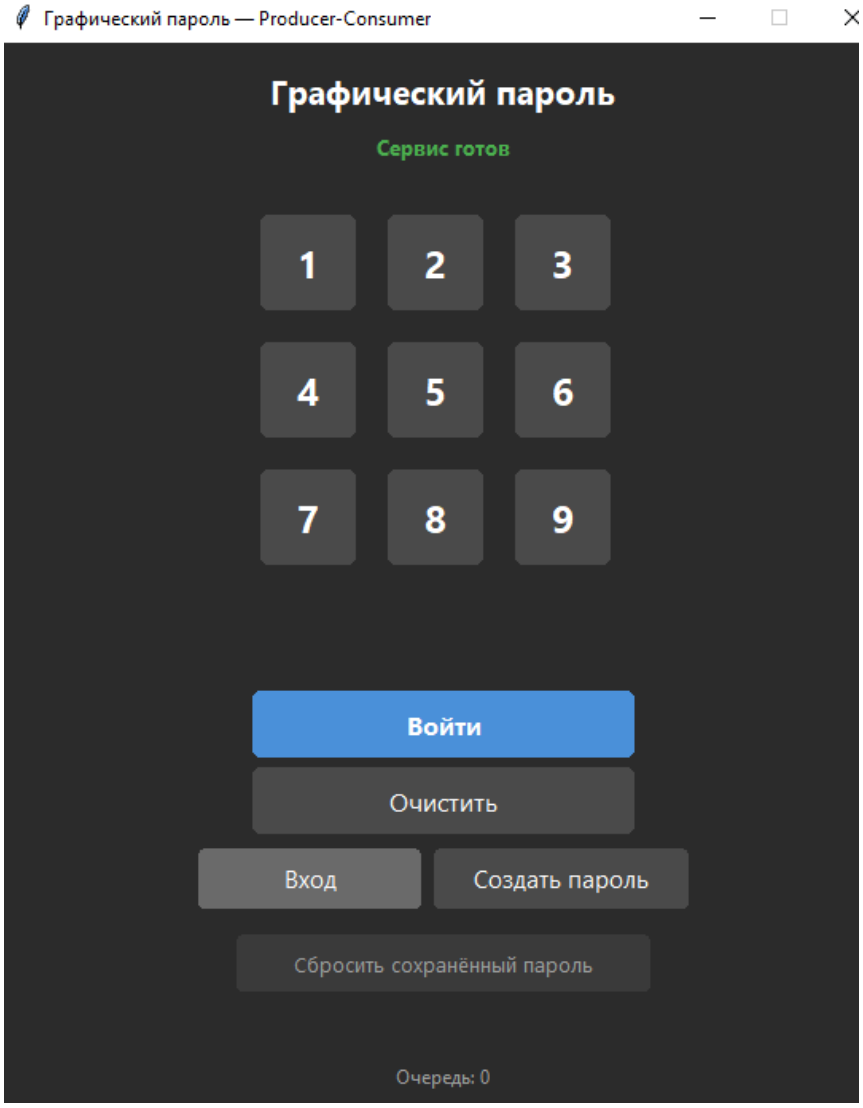


Рисунок 3 - Главное окно

Заголовок «Графический пароль», бейдж «Сервис готов» (зелёный), сетка 3×3, кнопки «Войти», «Очистить», переключатель «Вход / Создать пароль», «Сбросить сохранённый пароль», индикатор «Очередь: 0».

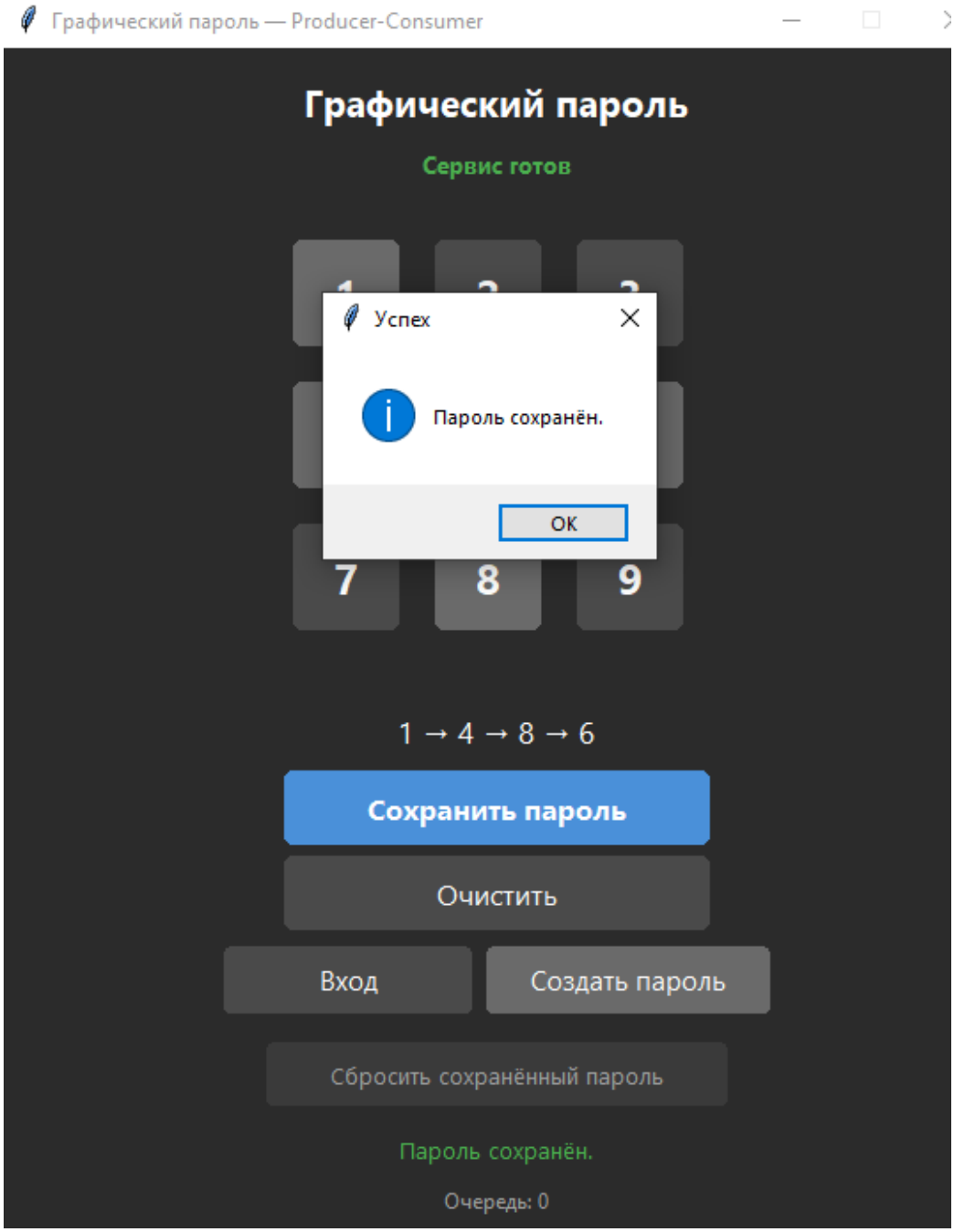


Рисунок 4 - Создание пароля

Выбрана последовательность 1 - 4 - 8 - 6, нажата «Сохранить пароль». Кнопка на мгновение меняет текст на «Обработка...», UI не «зависает» - сетка остаётся отзывчивой. По завершении - диалог «Пароль сохранён».

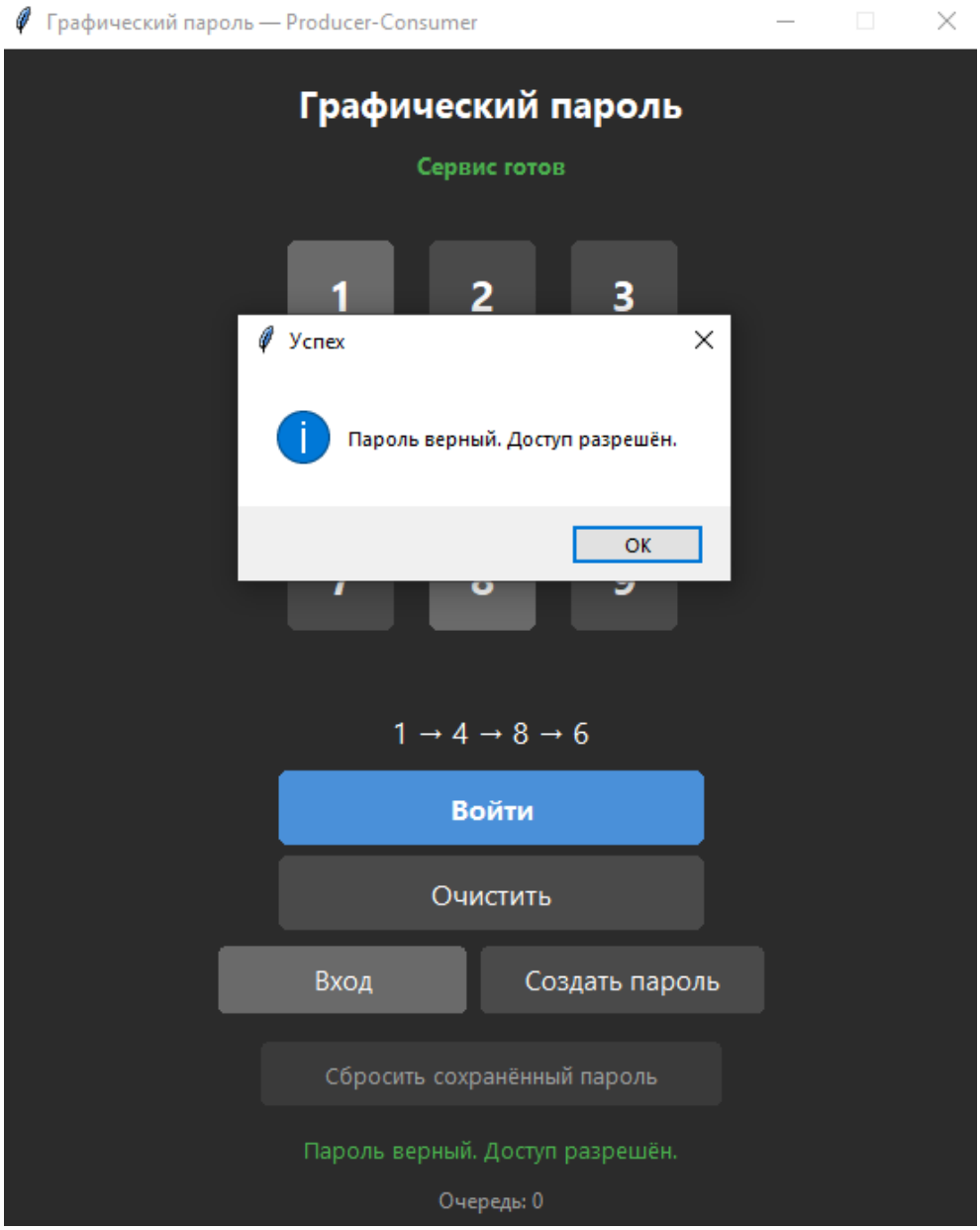


Рисунок 5 - Проверка правильного пароля

Введена та же последовательность, нажата «Войти». После короткой обработки - «Пароль верный. Доступ разрешён.».

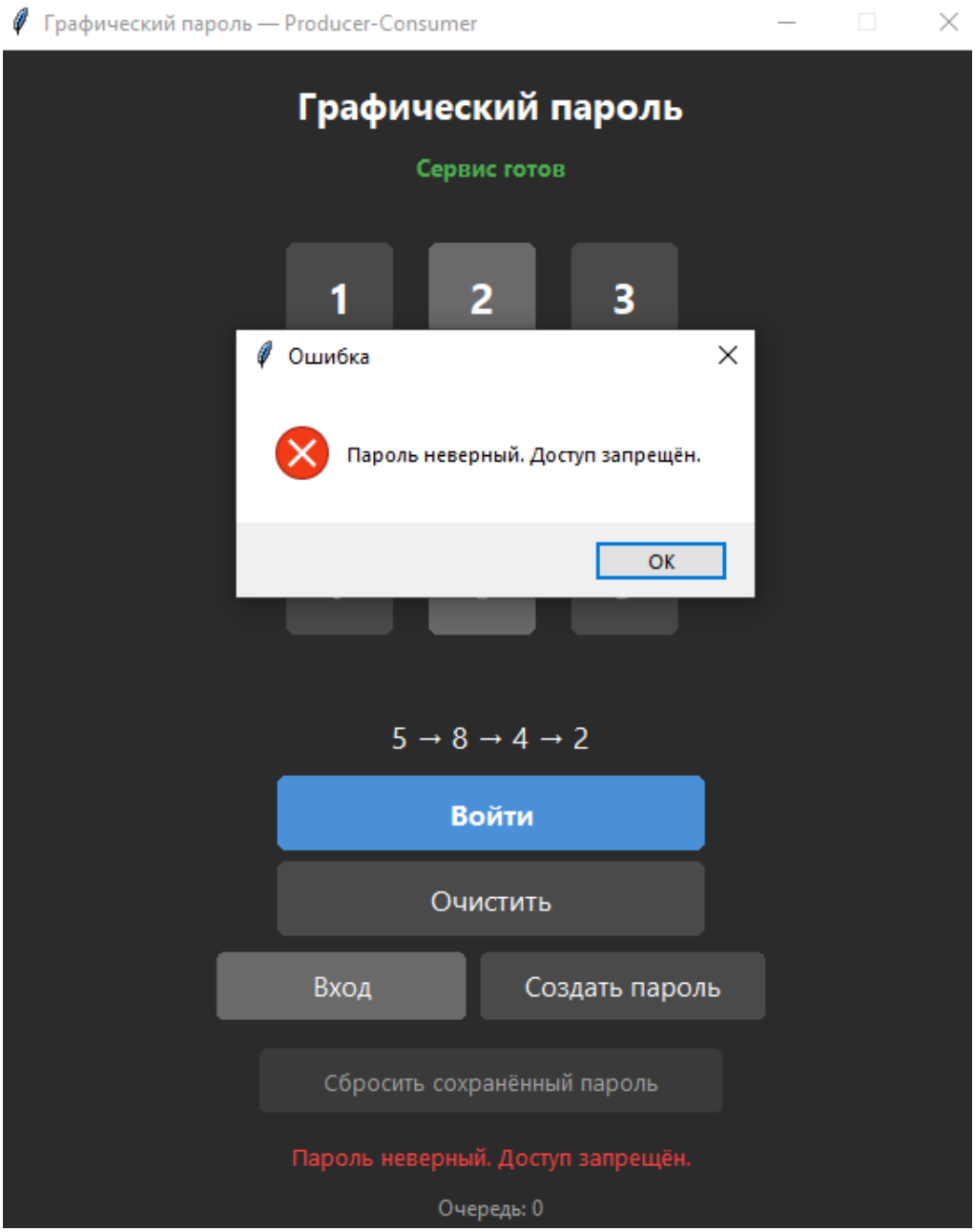


Рисунок 6 - Проверка неправильного пароля

Введена 5 - 8 - 4 - 2. Диалог «Пароль неверный. Доступ запрещён.».

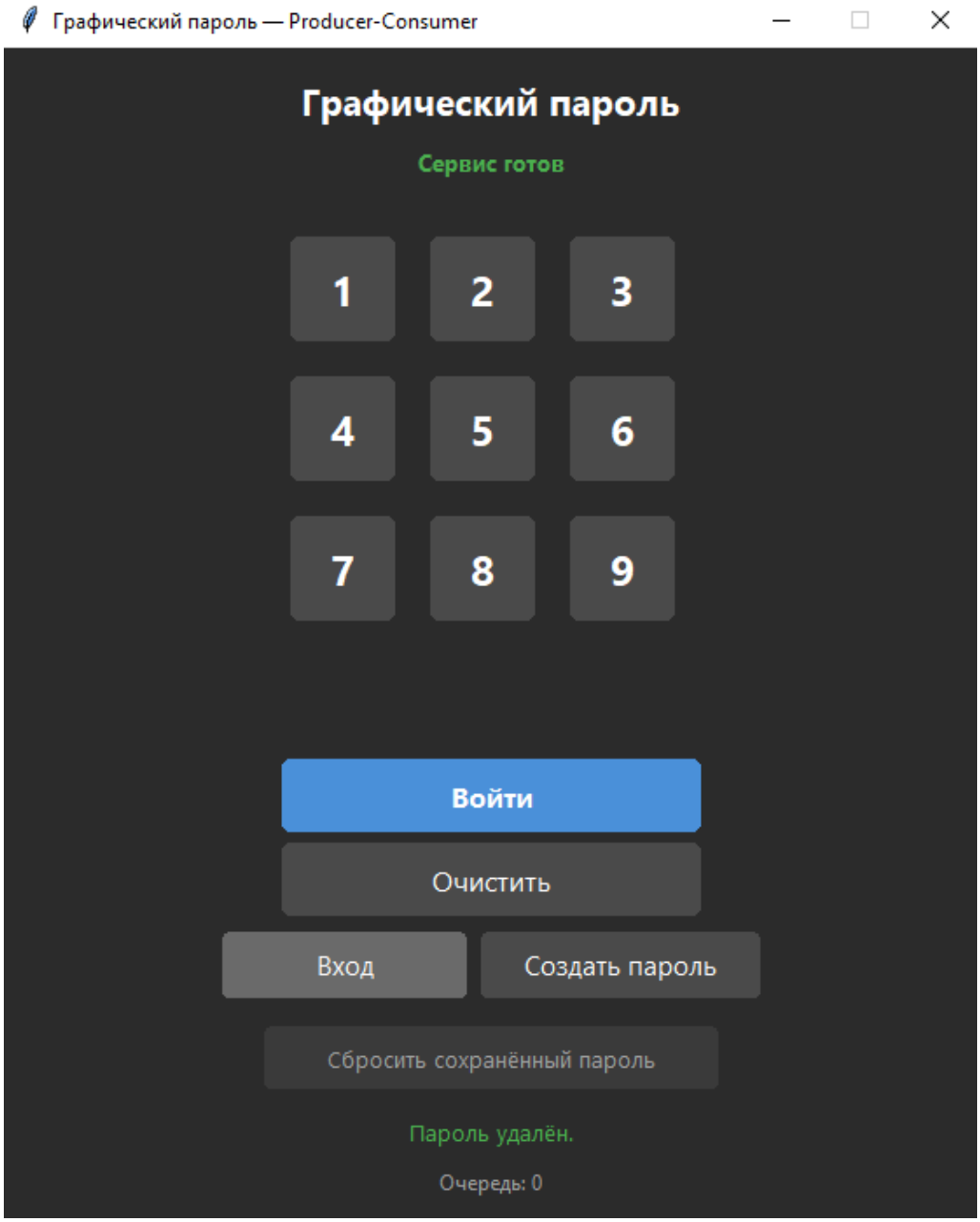


Рисунок 7 - Сброс пароля

Нажата «Сбросить сохранённый пароль». Диалог «Пароль удалён.», файл graphical_password.json удалён.

Вывод

В ходе выполнения лабораторной работы №15 был изучен и реализован идиоматический паттерн многопоточности Producer-Consumer. В приложении «Графический пароль» главный поток Tkinter выступает в роли производителя (producer), который генерирует задачи и помещает их в потокобезопасную очередь, а фоновый поток Worker выступает в роли потребителя (consumer), извлекая и обрабатывая эти задачи. Такое

разделение позволило полностью исключить зависание пользовательского интерфейса во время выполнения операций чтения файла, хэширования или записи данных. Взаимодействие между потоками безопасно организовано через модуль queue, а результаты возвращаются в UI через периодический опрос очереди с помощью метода root.after(). В итоге применение паттерна обеспечило высокую отзывчивость приложения и наглядно продемонстрировало эффективность асинхронной обработки задач при работе с графическим интерфейсом.